

AI Coding 开发过程规范 • v1.5

规定基于 O'CodingNavigator 的 AI 辅助软件开发两阶段过程、门禁判定与职责划分的要求。
使问题可判断、门禁可执行、为 AI 编码代理提供明确输入、为后续证据提供对照基线。

源版本：OCN-SPEC-001 v1.5（代替《AI Coding 最佳实践开发 SOP 两阶段》v8）

源文件：AI Coding 开发过程规范_v1.5_20260615.md

生成日期：2026-06-15 • 对应 OCN：o-coding-navigation@0.6.0-beta.0（SOP 0.5.0）

目录

TABLE OF CONTENTS

1	前言
2	引言
3	1 范围
4	2 规范性引用文件
5	3 术语和定义
6	4 过程模型
7	5 第一阶段要求（Planning Gate）
8	6 第二阶段要求（Execution Navigator）
9	7 门禁与状态推进
10	8 职责划分
11	9 不符合项与纠正措施
12	附录 A（规范性）操作规程
13	附录 B（资料性）修订历史

前言

本文件规定基于 O'CodingNavigator（以下简称 OCN）的 AI 辅助软件开发两阶段过程要求，属于工程过程规范。

本文件代替《AI Coding 最佳实践开发 SOP 两阶段》修订版 v8（2026-06-12）。与 v8 相比，本文件将原指导性文体转换为规范性文体；技术要求与 v8 等同，未新增、删除或修改任何功能性内容。历次版本变更情况见附录 B。

本文件由 Pudding Bot Data Science 起草并维护。主要起草人：Tim O。

本文件与 OCN 工具版本的对应关系如下：本规范 1.5 对应 SOP 0.5.0，发行载体为 npm 包 o-coding-navigation@0.6.0-beta.0（含 ocn 命令行工具与 ocn-mcp MCP 服务器）。本文件中引用的命令、退出码、检查项与文件路径均以该版本的实际行为为准；工具版本升级时，本文件应同步修订。

本文件的附录 A 为规范性附录，附录 B 为资料性附录。

引言

0.1 问题陈述

AI 辅助软件开发过程中存在一类系统性缺陷，本文件统称为虚假完成（false completion）：过程产物在形式上满足完成条件，而其所代表的工程事实并未成立。按判定对象的不同，虚假完成分为四类：

- **第一类（结构缺失型）**：产物的必需章节缺失或为空，而该产物被视为已完成。
- **第二类（逻辑未接线型）**：设计文档结构齐全，但系统的计算/决策语义——执行顺序、公式归属、分数分层、信号作用级别——未被显式定义为可校验结构，散落于多份文档的叙述文本中，导致实现阶段持续偏离设计。
- **第三类（角色盲区型）**：各文档门禁均已通过，但项目不满足必需验收角色的就绪条件，例如：无版本控制与 CI、无可复跑的测试命令、无可运维性责任人、无成本测算、无真实使用者。
- **第四类（实现缺位型）**：实施阶段的过程文档如实记录而实现并未发生。引擎仅校验过程文档自身的章节完整性、不校验文档所对应的现实，因此“本阶段无代码变更”的如实记录可通过全部门禁；其极端情形为全链空转，即整套过程的状态推进完成一轮，而任何实现变更从未被任何步骤排入。

0.2 方法

本文件采用的方法是可验证命题转换：将不可由机器判定的命题替换为可由机器判定的命题，并为每一类虚假完成配置对应的机械判定机制。

不可验证命题	替换后的可验证命题	判定机制
文档内容完整吗	必需章节是否全部存在且非空	章节门禁
逻辑接好了吗	计算/决策图是否命中六类硬缺陷	逻辑主干门禁
对验收角色准备好了吗	每个必需角色检查是否 PASS 或被显式豁免	就绪门禁
按计划实现了吗	每个任务规格冻结的验收命令是否以退出码 0 终止	任务门禁（任务规格与任务台账）

0.3 两阶段划分依据

软件开发过程的前段（问题定义至实施计划）具有天然顺序，适合线性推进与强门禁控制；后段（实施、验证至收敛判定）为非线性循环——代码修改、测试执行、缺陷修复、证据补充、PR 与 review 处理、收敛判断交替发生——不适合以线性文档推进为主要控制手段。后段最可靠的事实来源是既有的工程证据链（本地 git、GitHub PR、commit、diff、review、CI、测试结果、验收证据映射、验证汇总），而非事后补写的记录。本文件据此将开发过程划分为两个连续阶段，分别规定其控制方式。

1 范围

本文件规定了基于 OCN 的 AI 辅助软件开发两阶段过程的过程模型、第一阶段（Planning Gate）文档与门禁要求、第二阶段（Execution Navigator）证据与任务循环要求、门禁判定与状态推进规则、AI 编码代理与操作者之间的职责划分，以及不符合项的纠正措施。

本文件适用于使用 OCN 0.5.0 及兼容版本组织 AI 辅助软件开发的项目，包括单人项目、小团队项目与平台级项目（分级要求见 5.4.3）。

本文件不规定具体编程语言、代码风格或测试框架的选型。

2 规范性引用文件

下列文件中的内容通过文中的规范性引用而构成本文件必不可少的条款。

- o-coding-navigation@0.6.0-beta.0（npm 发行包，含 ocn 命令行工具与 ocn-mcp MCP 服务器）

- OCN 文档模板体系 docs/00 至 docs/19（随 SOP 0.5.0 分发的产物模板与必需章节定义）
- OCN 治理记录体系：DEC（决策记录）与 AM（修正案）——对冻结设计契约的偏离以修正案记录，修正案对其所记录的偏离具有规范效力
- docs/quickstart.md（OCN 安装步骤、初始化后文件树与排障说明）

3 术语和定义

下列术语和定义适用于本文件。

3.1 两阶段过程（two-phase process） 由第一阶段（3.2）与第二阶段（3.3）顺序衔接构成的开发过程结构，衔接点为 11 号文档（build plan，见 4.2）。

3.2 第一阶段；Planning Gate 从 00 号文档至 11 号文档的线性推进阶段，以强门禁与结构化定义为控制方式，目标是形成可执行的实施计划。

3.3 第二阶段；Execution Navigator 11 号文档门禁通过后进入的实施阶段，以工程证据链与任务台账为控制方式，目标是形成证据充分的实现、验证与最终判定。

3.4 门禁（gate） 在产物或状态推进路径上设置的机械判定点；判定不通过时，过程不应继续推进。

3.5 状态推进（advance） 通过 ocn advance 命令将项目状态游标移至下一步骤的操作；执行时自动先行运行门禁判定。

3.6 虚假完成（false completion） 过程产物在形式上满足完成条件而其所代表的工程事实未成立的缺陷，分四类，见 0.1。

3.7 全链空转（full-chain idle run） 第四类虚假完成的极端情形：整套过程的状态推进完成一轮而无任何实现变更被排入。

3.8 逻辑主干（Logic Backbone） 以有类型、有角色的有向图（DAG 与 DMN 决策分层）显式表达系统计算/决策语义的设计层产物，对应文件 docs/07-logic-backbone.md，可被机器校验，见 5.2。

3.9 就绪主干（Readiness Backbone） 基于角色编目的横切门禁规则集，在每次门禁判定中于章节检查与逻辑主干检查之后自动运行，贯穿两个阶段，见 5.4。

3.10 任务主干（Task Backbone） 由任务规格（3.11）、任务台账（3.12）与验收命令冻结机制构成的实施控制结构，见 5.3。

3.11 任务规格（task specification） build plan 中机器可解析的任务定义块，每个任务构成一份可证伪的最小规格，含必填字段 goal、traces、touches、verify、dod 与可选字段 depends、phase、timeout，见 5.3。

3.12 任务台账（task ledger） 11 号文档门禁通过时由引擎生成的任务清单文件 .ocoding/task-ledger.json，记录各任务及其哈希冻结的验收命令与完成状态。

3.13 豁免 (waiver) 对确实不适用于当前项目的就绪检查所作的显式免除，应附理由与探针 (waive-with-probe)，语义见 5.4.5。

3.14 探针 (probe) 用于持续验证豁免前提是否成立的确定性命令；授予豁免时探针应通过，此后每次门禁运行时复验。

3.15 分级 (tier) 项目按规模声明的就绪要求等级，取值为 solo、team、platform，决定必需就绪检查的范围，见 5.4.3。

3.16 就绪检查 (readiness check) 对应某一验收角色关注点的可证伪检查项，判定结果为 PASS、FAIL、UNKNOWN、WAIVED 或 NA。

3.17 开放世界判定 (open-world evaluation) 就绪检查的判定语义：无证据 (UNKNOWN) 与失败 (FAIL) 同样阻断；缺少记录不视为通过。

3.18 哈希冻结 (hash freeze) 将命令或配置以哈希方式固定写入受控文件的机制；被冻结内容在实施期间被修改时，应重新通过门禁方可生效。

3.19 证据 (evidence) 可由机器读取的工程事实，包括 git 状态、commit、diff、PR 元数据、checks、review 结论、测试结果及其与验收标准的映射。

3.20 操作者 (operator) 对项目结果负责并保留裁定职责的人员。

3.21 AI 编码代理 (AI coding agent) 执行文档起草、代码实现、证据归纳等任务的 AI 系统，实例包括 Claude Code、Codex、LFG 等。

3.22 回拨 (rewind) 通过 `ocn rewind` 命令将状态游标移回当前 SOP 版本声明序中严格更早步骤的受控操作；执行须给出理由，操作及理由写入审计记录，不附带任何门禁豁免，见 7.3。

3.23 重开循环 (cycle) 通过 `ocn cycle new` 命令将本轮运行时状态归档并自首步骤开启新一轮的受控操作；文档产物保留，审计记录跨轮连续，见 7.3。

3.24 自动模式 (auto mode) 由操作者通过 `ocn auto` 命令按阶段显式开启的可选运行模式，开启后 AI 编码代理在被授权的阶段内可不经逐步人工确认而自主触发状态推进与任务勾销等受控操作；默认关闭（手动模式）。自动模式委托的是触发权而非裁定权——门禁与冻结验收命令的判定不因开启而豁免，见 7.6。

3.25 调用方身份 (actor) 命令调用方的身份标记，取值为 user（人）或 ai_agent（AI 编码代理），由环境变量 `OCN_ACTOR` 或 `--actor` 参数解析；写入审计记录，用于区分人工操作与代理操作。该标记为治理签名而非安全边界，见 7.6。

3.26 熔断 (circuit breaker) 自动模式下的失控保护：AI 编码代理在同一步骤连续触发门禁失败达到阈值（默认 5）时，引擎自动暂停自动模式并拒绝该代理的后续受控操作，人工操作不受影响；解除须由操作者执行 `ocn auto resume`，见 7.6。

3.27 决策痕迹 (decision trace) 自动模式下对 AI 每次受控操作的可复盘记录，含代理自述理由（`--rationale`，应写明背景、依据与操作）与引擎独立记录的机器上下文（门禁结果、任务台账摘要、冻结命令与耗时），见 7.6。

3.28 独立评审 (independent review) 自动模式下, AI 编码代理在被授权阶段内行使任一受控触发前, 委派的一次独立、全新上下文的实现评审: 由评审子代理以资深领域专家视角, 依据本步骤要求、可追溯验收准则、当次改动差异与本规范, 给出 PASS/FAIL 结论与具体问题。其性质为替代被省略人工评审的尽职调查, 而非门禁——裁定权仍属门禁与冻结验收命令, 见 7.6.8。

4 过程模型

4.1 两阶段结构

开发过程应划分为两个连续阶段。

第一阶段 (Planning Gate) 覆盖 00 号至 11 号文档, 应具备下列特征:

- 线性推进;
- 强门禁;
- 强结构化;
- 以文档与定义为主要产物;
- 目标为形成可执行的 build plan。

第二阶段 (Execution Navigator) 自 11 号文档门禁通过后开始, 应具备下列特征:

- 非线性循环;
- 以证据链为主要事实来源;
- 以状态判定与下一轮行动建议为主要输出;
- 不以手工推进线性文档为核心控制手段;
- 目标为形成证据充分的实现、验证与最终判定。

4.2 衔接点

docs/11-build-plan.md 为两阶段的衔接点。该文档门禁通过表明:

- 第一阶段的定义已足以支撑实施开工;
- 任务台账已生成且各任务验收命令已冻结 (见 5.3.5);
- 过程控制方式由文档推进转换为证据读取、偏差控制与收敛促成。

4.3 工具角色

第一阶段中, OCN 承担 Planning Gatekeeper 职能, 负责判定: 是否允许进入下一步骤; 当前文档是否具备最低完整性; 哪些定义缺失; 是否具备开工条件。

第二阶段中, OCN 承担 Execution Evidence Navigator 职能, 负责判定: 当前实现进展; 已具备与缺失的证据; 当前应继续开发、请求修改、进入 review 或等待人工判定; 下一轮 AI 编码代理应执行的任务。

两阶段的状态推进默认由操作者执行 (手动模式)。操作者可按阶段开启自动模式, 将被授权阶段内的推进触发委托给 AI 编码代理; 此时 OCN 的判定职能不变, 改变的仅是触发主体, 且受熔断与人工专属禁区约束, 见 7.6。

4.4 过程闭环

完整过程闭环为: 第一阶段完成 00 至 11 号文档并冻结任务台账; AI 编码代理按任务规格实现, git 与 GitHub 形成证据链; OCN 读取证据、生成下一轮任务简报、汇总验证状态与判定草案; 操作者裁定 review、merge 与发布。

4.5 文档的作用界定

文档的作用在于: 使问题可判断、使门禁可执行、为 AI 编码代理提供明确输入、为后续证据提供对照基线。不能服务于后续实现与验证的文档不应纳入过程要求。第二阶段中, 过程记录不构成实现发生的证据; 实现发生与否应以任务台账与验收命令的执行结果判定 (见 5.3、7.3)。

5 第一阶段要求 (Planning Gate)

5.1 文档序列总则

第一阶段应按 00 至 11 的顺序逐份完成下列文档。每份文档应自模板创建 (ocn doc create <type>), 并通过门禁判定 (ocn check) 后方可推进 (ocn advance)。操作规程见附录 A.3。

各文档的目的与内容要求如下:

5.1.1 00 项目简报 (docs/00-project-brief.md, type project-brief) 应定义问题、目标、用户与成功标准。该层定义不清时, 后续全部内容将发生偏离。

5.1.2 01 范围 (docs/01-scope.md, type scope) 应明确范围内事项 (in scope)、范围外事项 (out of scope)、技术约束与本轮完成边界, 用于控制范围扩张。

5.1.3 02 PRD (docs/02-prd.md, type prd) 应将产品需求转换为结构化功能描述, 明确对象、行为、输入、输出与边界, 为后续验收与实现提供对照基线。

5.1.4 03 验收标准 (docs/03-acceptance-criteria.md, type acceptance-criteria) 应将目标转换为可判定的验收项, 使“已实现”成为可判断命题, 并为第二阶段的证据映射 (ocn evidence map) 提供基线。

5.1.5 04 技术架构 (docs/04-technical-architecture.md, type technical-architecture) 应

确定最终技术选择，明确运行形态、语言、存储、部署、约束与风险，形成架构决策锚点。

5.1.6 05 信息架构 (docs/05-information-architecture.md, type information-architecture) 应定义系统的信息结构。

5.1.7 06 数据模型 (docs/06-data-model.md, type data-model) 应定义系统的数据结构。

5.1.8 07 逻辑主干 (docs/07-logic-backbone.md, type logic-backbone) 应满足 5.2 的全部要求。

5.1.9 08 接口契约 (docs/08-api-contract.md, type api-contract) 应定义接口契约；接口暴露的内容应为逻辑主干所定义的计算结果。

5.1.10 09 测试策略 (docs/09-test-strategy.md, type test-strategy) 应定义测试策略；测试对象应覆盖逻辑主干所定义的计算/决策图。

5.1.11 10 MVP 计划 (docs/10-mvp-plan.md, type mvp-plan) 应界定最小可行交付的范围与排序。

5.1.12 11 Build Plan (docs/11-build-plan.md, type build-plan) 应将第一阶段的定义压缩为可执行计划，为执行阶段提供任务切分依据、为证据分析提供对照表，并满足 5.3 规定的任务规格要求。该文档为第一阶段终点与第二阶段入口。

5.2 逻辑主干要求

5.2.1 书写位置

逻辑主干应在 06 数据模型完成之后、08 接口契约与 09 测试策略之前书写。该位置由依赖关系决定：逻辑主干的输入与分数由数据模型的字段计算得出；接口契约暴露其计算结果；测试策略以其图结构为测试对象。文件号 07 即书写顺序（第 7 份产出），自 08 接口契约起的文档编号相应顺移。第一阶段结束前，六类硬缺陷（5.2.4）应清零。

5.2.2 图模型

逻辑主干应将系统的计算/决策语义建为有类型、有角色的有向图（DAG 与 DMN 决策分层），构成要素如下：

要素	取值	说明
节点 kind	input / formula / score / judgment / signal	输入（指标） / 公式 / 分数 / 判断 / 信号
节点 role（必填）	input / intermediate / terminal_explanatory / trigger / hint	输入 / 中间（应被消费） / 终点（仅解释） / 触发（驱动功能） / 提示（仅告知）
边（上游指向下游）	feeds / serves / triggers / explains	feeds 表达计算顺序与分数向下层传递；serves 表达公式服务于判断；triggers 表达信号驱动功能；explains 表达仅解释、仅提示

逻辑主干应显式回答四项语义：执行顺序（先算什么、后算什么）；公式归属（哪个公式服务哪个判断）；分数分层（哪个分数进入下一层、哪个分数仅作解释）；信号作用级别（哪个信号驱动功能触发、哪个信号仅作提示）。

5.2.3 单一事实源要求

逻辑主干不应以叙述文本替代图定义；其图定义为系统计算/决策语义的单一事实源，应可被机器校验。

5.2.4 六类硬缺陷判定

ocn check 应对逻辑主干判定下列六类硬缺陷，命中任一项即判定不通过（退出码 2，逐条列明）：

1. 缺角色（节点未声明 role）；
2. 重复节点 id；
3. 悬空引用（边指向未定义节点）；
4. 依赖环（feeds/serves/triggers 子图成环）；
5. 孤儿节点（input 或 intermediate 节点无下游消费）；
6. 未绑定触发（trigger 节点无 triggers 边指向已定义目标）。

5.2.5 机器投影

逻辑主干门禁通过后，引擎应将规范化的图写入 .ocoding/logic-graph.json；ocn brief 应注入执行顺序与触发清单摘要，供第二阶段对照，防止实现偏离。

注：本机制在 OCN 0.3.0 中产品化为 artifact_logic_backbone。范式参考：dbt 的 ref-DAG、DMN 决策需求图、Event Modeling、架构适应度函数。

5.3 任务规格要求

5.3.1 章节要求

自 SOP 0.5.0 起，build plan 应包含 ## Task Specs | 任务规格章节，将实施拆分为任务规格块；每个任务构成一份可证伪的最小规格。

5.3.2 块格式

任务规格块的格式如下例（示例为说明性内容）：

```
### task_phase0_runtime_skeleton
- goal: permit runtime 最小可运行骨架（替换全部 NotImplementedError）
- traces: AC-03, AC-07
```

- touches: score_risk
- verify: pytest tests/test_runtime.py -q
- dod: 接口全部可执行; RED→GREEN 过程记入 12 号

5.3.3 字段表

字段	必填性	含义与约束
goal	必填	任务目标的单句陈述
traces	必填	应解析到 03 验收标准中真实存在的 AC 编号
touches	必填	应引用逻辑主干中已定义的节点; 悬空即阻断
verify	必填	本任务专属的确定性验收命令
dod	必填	完成定义 (Definition of Done)
depends	可选	任务依赖; 不应成环
phase	可选	任务分组
timeout	可选	验收命令执行秒数上限

5.3.4 六类硬缺陷判定

build plan 门禁应对任务规格判定下列六类硬缺陷, 命中任一项即判定不通过 (退出码 2):

1. 重复或非法任务 id;
2. 必填字段缺失;
3. traces 悬空 (引用不存在的 AC);
4. touches 悬空 (引用不存在的逻辑主干节点);
5. depends 悬空或成环;
6. 零任务 (章节存在但无任务拆分)。

5.3.5 任务台账与冻结

11 号文档门禁通过时, 引擎应生成任务台账 .ocoding/task-ledger.json, 并将每个任务的验收命令以哈希方式冻结写入台账。验收命令不应处于实施写路径之上; 实施期间修改 build plan 应重新通过门禁方可生效。

任务完成状态仅应由该任务冻结的验收命令以退出码 0 终止的事实确立 (ocn task check); 不应提供人工标记完成的途径。任务台账存在未完成任务时, 状态推进不应允许离开 BUILD 状态 (state_build)。

注: 本机制在 OCN 0.5.0 中产品化为 task 门禁与 ocn task 命令。

5.3.6 拆分准则

一个任务宜对应一次 PR 级增量 (diff ≤ 500 行), 并应具备独立可执行的验收命令。拆分质量由操作者与 AI 编码代理在书写 build plan 时把控; 门禁仅对 5.3.4 所列硬缺陷作判定。

5.4 就绪要求

5.4.1 性质与运行时机

就绪主干不是一份文档，也不是一个步骤，而是随 SOP 打包的横切门禁规则集。其应在每次 ocn check、ocn gate、ocn advance 执行时，于章节门禁与逻辑主干门禁之后自动运行，贯穿两个阶段。其判定对象为第三类虚假完成（角色盲区型，见 0.1）。

注：本机制自 OCN 0.4.0 起产品化为 readiness 横切门禁。设计参考：readiness review、Definition of Done、生产就绪评审（PRR）。

5.4.2 角色编目

“缺了哪些维度”不可穷举；“哪些角色必须验收”有边界且可编目。就绪检查的角色集取自外部成熟编目（基于 APQC PCF 与 ITIL 的 IT 角色知识库），共 54 个角色，按职能分四层；每个角色至少对应一条验收关注点，合计 55 条可证伪的就绪检查：

层	角色数	典型角色	典型检查关注点
战略层 (strategy)	10	CIO、IT 战略规划、业务关系经理、企业架构师	真实使用者、价值假设、成本测算、过度准备
架构层 (architecture)	8	解决方案架构师、数据架构师、安全架构师	架构决策锚点、数据模型与接口契约一致性、安全基线
交付层 (delivery)	15	开发、QA、DevOps、业务分析、项目经理	版本控制与 CI、测试可复跑、验收标准可证伪、变更证据
运营层 (operations)	21	SRE、服务台、安全运营、容量/可用性管理	可运维性责任人、监控与告警、故障与回滚路径

5.4.3 分级 (tier)

并非每个项目均需满足全部 54 个角色的检查。每条检查声明其在哪些 tier 下为必需（由该角色通常出现的最小团队规模推导）；项目应在初始化时通过 ocn init --tier <t> 定级：

init --tier	就绪 tier	适用范围	必需检查范围
minimal	solo	单人或极小项目	最小集：开发、QA、DevOps、基本安全与使用者等核心角色
production	team	小团队、面向生产	solo 全集，另加项目管理、架构、成本、合规等团队级角色
full	platform	平台级、多团队	全部 54 角色，含治理、容量、连续性等平台级角色

不属于当前 tier 的检查应自动判定为 NA（不计缺漏、不阻断）；属于当前 tier 的检查方参与门禁。tier 应在初始化时随探针命令一并哈希冻结（R4）；实施中途调低定级以规避检查的行为应被检出。

5.4.4 取证与判定

就绪检查的取证与判定应满足下列要求：

- **确定性取证**：通过文档别名 (doc slug 通配) 与仓库探针 (文件存在性、config.yaml 中登记的 build/test 命令) 解析证据；不应调用 LLM；本地证据优先。
- **开放世界判定**：block 级且当前 tier 必需的检查，仅 PASS 或 WAIVED 视为通过；FAIL 与 UNKNOWN 同样阻断。阻断时应返回 ERR_GATE_FAILED (退出码 1)，并逐条给出双语修复提示 (fix_hint)。
- **warn 级检查**仅进入 ocn brief 提示，不阻断 (例如战略层的过度准备检查)。
- **判定台账**：每轮评估应写入 .ocoding/readiness.json；ocn brief 应将未决项及其 fix_hint 列为工作清单。

5.4.5 豁免条件

确实不适用于当前项目的检查应通过有条件豁免 (waive-with-probe) 处理，其语义如下：

- 授予豁免时探针应通过；不应授予无探针验证的豁免；
- 此后每次门禁运行时应复验探针；
- 项目离开当前状态时豁免自动过期；
- 豁免为操作者专属操作，不应暴露给 MCP 接口；每次授予应写入审计记录。

豁免命令与使用时机见附录 A.4。

6 第二阶段要求 (Execution Navigator)

6.1 证据来源

第二阶段的事实来源应为工程证据链，包括：

- 本地 git 状态、当前分支、commit 历史、改动文件；
- GitHub PR 元数据、checks 状态、review 结论；
- 验收证据覆盖 (acceptance evidence coverage)；
- 验证汇总 (verification summary)。

第二阶段不应以重复手工记录替代上述既有证据。

6.2 任务循环

自 SOP 0.5.0 起，BUILD 状态的核心节奏应为逐个完成任务台账中的任务：

1. `ocn task list`——查看台账中各任务状态及下一任务；
2. 派发任务（已接线 Claude Code 时使用 `/ocn-next`，任务目标为任务规格原文）；
3. AI 编码代理按测试驱动方式实现，修改范围应限于该任务的 `touches` 所引用节点对应的实现；
4. `ocn task check`——执行该任务冻结的验收命令，以退出码 0 确立完成状态。

上述循环应重复执行至台账全清；台账未清时状态推进不应放行（见 7.3）。

6.3 证据循环

每轮迭代宜按下列顺序执行（命令功能定义见 6.4，操作规程见附录 A.5）：

1. `ocn exec status`——读取本地 git 现场；
2. `ocn github analyze-pr <n>`——存在 PR 时读取 PR 元数据、checks 与 reviews；
3. `ocn evidence map [--pr <n>]`——对照 03 验收标准映射证据；
4. `ocn next-prompt`——生成下一轮 AI 编码代理任务简报；
5. AI 编码代理执行一轮实现；
6. `ocn verify status --mode combined --pr <n>`——汇总验证就绪度；
7. `ocn verdict draft`——草拟阶段判定。

已接线 Claude Code 时（见 7.5）：`/ocn-next` 替代第 1 至第 4 步（`next-prompt` 内部读取 git、state 与验收证据；第 1、3 步降级为操作者主动查证工具）；第 5 步期间的编辑反馈与回合结束门禁由钩子自动执行；第 6、7 步为操作者收口工具，在任何模式下均不被替代。

6.4 只读证据命令功能定义

下列六条命令均应为只读操作：不写入 `.ocoding/`、不改变 git 与 gh 状态、不调用 LLM。

命令	功能	使用时机
<code>ocn exec status</code>	读取本地 git 证据（分支、head、工作区脏/净、改动文件、近期 commit）与当前 OCN 状态	每轮迭代开始时；中断后恢复现场时
<code>ocn github analyze-pr <n></code>	只读分析 PR 元数据、files changed、checks、reviews，并给出风险标记	PR 创建后；review 前；merge 决策前
<code>ocn evidence map [--pr <n>]</code>	将 03 验收标准与本地及 PR 证据逐条映射，输出 evidence-found / candidate / missing / needs-human-review 四种状态	需要判定实现完成度的任何时刻
<code>ocn next-prompt</code>	基于状态、git 与验收证据生成下一轮 AI 编码代理任务简报（目标、允许的工作、禁止动作、验证命令、停止条件）	向 AI 编码代理派发任务之前
<code>ocn verify status</code>	读取 package scripts 与验证信号，汇总为 ready / partial / blocked / pending	每轮实现结束后；仅当结果为 ready 时宜进入 review

命令	功能	使用时机
ocn verdict draft	基于证据草拟阶段判定（继续开发 / 请求修改 / 可 review / 可 merge / 待人工）；结论交由操作者裁定	里程碑收口时

6.5 文档角色转变

12 至 19 号文档在第二阶段可继续存在，但其角色应由“操作者手工推进的表单”转变为：

- 对执行证据的汇总视图；
- 对 GitHub、git、CI 状态的结构化归纳；
- 对最终判定（verdict）的人类可读报告。

第二阶段为证据驱动、文档承接；不应为文档驱动、证据补写。文档清单如下：

编号	文档
12	implementation log（实现日志）
13	change evidence（变更证据）
14	integration notes（集成说明）
15	verification report（验证报告）
16	acceptance mapping（验收映射）
17	failure fix log（故障修复日志）
18	regression evidence（回归证据）
19	final build verdict（最终构建判定）

00 至 11 号文档构成强门禁输入体系；12 至 19 号文档构成执行证据承接体系。

7 门禁与状态推进

7.1 门禁组成与顺序

ocn check 应按下列顺序执行三道门禁：

1. 章节门禁——校验必需章节的存在与非空；
2. 逻辑主干门禁——校验 5.2.4 所列六类硬缺陷（07 号文档就位后）；
3. 就绪门禁——按 5.4 执行就绪检查。

任务规格校验（5.3.4）属于 11 号文档的产物校验，归入第 2 类判定（退出码 2）。

7.2 退出码语义

ocn 命令的退出码语义应符合下表；该语义为机器可读的稳定契约：

退出码	语义	错误码	典型情形
0	通过	OK	门禁通过；命令正常完成
1	门禁未通过	ERR_GATE_FAILED	就绪检查存在 FAIL 或 UNKNOWN 的必需项
2	产物缺失或无效	ERR_ARTIFACT_INVALID	必需章节缺失；逻辑主干命中六类硬缺陷；任务规格命中六类硬缺陷
4	配置、锁或 IO 错误	ERR_IO_OR_CONFIG	配置文件损坏；锁获取失败
5	SOP 版本不兼容	ERR_SOP_VERSION	项目锁定的 SOP 与工具内置版本不兼容

被阻断时的处置规则：退出码 2 应修订产物本身（补章节、修逻辑主干、修任务规格）；退出码 1 应执行 `ocn readiness list` 查看阻断项与 `fix_hint`，补充证据或在确实不适用时显式豁免 (5.4.5)。

7.3 推进规则

状态推进应满足下列规则：

- `ocn advance` 执行时应自动先行运行门禁；门禁未通过时不应推进。
- 第一阶段应按顺序线性推进：未完成定义不应进入设计；未确立验收标准不应进入实现；未确定架构不应拆分数据与接口；未确定测试策略不应进入 BUILD。
- 每道门禁在章节检查与逻辑主干检查之后应运行就绪检查：当前 tier 必需的 block 级检查应全部 PASS 或被显式豁免，否则 `advance` 不应放行。补充证据与显式豁免为仅有的两条处置路径。
- **任务台账存在未完成任务时，`advance` 不应允许离开 `state_build`。**
- 状态推进不构成义务。当现实证据不满足要求时，操作者不应执行 `advance`；状态游标可驻留原地直至证据补齐。
- 第二阶段不应以 `advance` 为主要交互方式，应以 6.2 任务循环与 6.3 证据循环为主。
- 状态游标的回退应仅通过 `ocn rewind --to <stepId> --reason <text>` 执行：目标步骤应存在于当前锁定 SOP 版本且在声明序中严格早于当前游标；理由为必填项并写入审计记录；回拨不修改任何文档产物，亦不附带门禁豁免——回拨后的每次 `advance` 应重新运行全部门禁。
- 到达终点步骤后开启下一轮迭代应仅通过 `ocn cycle new --yes` 执行：本轮运行时状态应归档至 `.ocoding/cycles/` 下的轮次目录，文档产物保留，SOP 锁定版本不变，审计记录不随轮归档、应跨轮连续。
- 不应通过直接编辑 `.ocoding/state.json` 移动状态游标；该行为绕过锁保护与审计记录，构成不符合项（NC-10）。
- 完成边界（见 1 范围对应的项目范围文档）由多个里程碑构成的项目，宜以**里程碑循环**组织各里程碑的衔接：每一里程碑判定完成后，将游标回拨至 `build plan` 步骤，在保留既有任务规格原文（验收命令文本不变）的前提下追加下一里程碑的任务规格，并重新通过门禁。任务台账按“任务标识与验收命令哈希均不变”的规则保留已完成状态，由此构成项目级累积台账；已完成任务的验收命令文本被修改时，该任务应重置为未完成。重

开循环（ocn cycle new）宜仅在完成边界达成后使用，用于归档本轮并开启下一项目周期；轮内的逐里程碑迭代不应使用重开循环。

7.4 第二阶段判定结论

第二阶段的阶段判定应从下列五种结论中选取，判定条件如下：

结论	判定条件
继续开发	证据不足、验证未完成或实现未收敛
请求修改	证据已足以指出明确缺口、失败或不一致
进入 review	主要验证通过、证据基本具备，尚需人工审查
准备 merge	PR 干净、checks 全部成功、证据充分、无阻断项
等待人工判定	存在定性标准、多义风险或冲突证据

evidence-candidate 状态不应被视同 evidence-found；证据不足时应维持保守判定。verify status 为 partial 时不应进入 merge；应先补证据、再补验证、再判定是否 ready。

7.5 Claude Code 钩子行为

ocn agent setup 应生成下列集成文件（命令幂等；--force 仅应用于修复损坏的 settings.json）：

- .claude/settings.json，含两条钩子：
 - **Stop 钩子**：AI 编码代理结束回合时自动运行 ocn check；检查未通过时应将修复提示返回代理会话以继续修正，不应允许回合在门禁未通过的情况下结束；
 - **PostToolUse 钩子**：每次文件编辑后自动运行 config.yaml 中登记的 commands.lint 与 commands.typecheck，并将错误输出反馈至代理会话即时修正；
 - 两条钩子均应带 command -v ocn 守卫；未安装 ocn 的环境中钩子不产生作用。
- .claude/ocn.md：治理契约，随每个代理会话自动加载；
- /ocn-next 斜杠命令：第一阶段中自动完成 brief 与 doc create 并开始执行；第二阶段中派发下一个未完成任务（任务目标为任务规格原文）。

上述文件提交入库（git add .claude CLAUDE.md）后，克隆该仓库的全部成员即获得相同接线，无须单独配置。接线完成后，操作者在每个任务中保留的动作为 /ocn-next 与 ocn advance 两项；其余纪律由钩子机械执行。无法机械强制的纪律不应作为过程依赖。

7.6 自动模式（可选）

状态推进与任务勾销默认为操作者专属（手动模式）。操作者可按阶段将其触发委托给 AI 编码代理，规则如下。

7.6.1 性质

自动模式委托触发权而非裁定权。开启后，每次 `ocn advance` 仍应完整运行 7.1 的三道门禁；任务完成仍应仅由冻结验收命令以退出码 0 确立（5.3.5）。门禁栈、逻辑主干判定、就绪判定与任务台账判定均不因开启自动模式而豁免。

7.6.2 授权开关

自动模式应仅通过 `ocn auto` 命令开启与关闭：`ocn auto on --phase <1|2|all>`、`ocn auto off [--phase]`、`ocn auto resume`、`ocn auto status`、`ocn auto trace`。`--phase` 为开启时的必填项，避免误开全自动。该开关本身为操作者专属，应拒绝 `ai_agent` 调用；开关动作（`on/off/resume`）与熔断暂停写入审计（事件 `auto_mode_changed`）；`status` 与 `trace` 为只读，不写审计。授权状态写入用户所有的 `.ocoding/config.yaml` 的 `automation` 块；该块缺失或损坏时应判为全手动（fail-safe 方向）。

7.6.3 阶段委托范围

阶段归属按 `advance` 的目标状态判定：

- **第一阶段（phase 1）**：目标状态属 `DISCOVERY`、`SPEC`、`DESIGN`、`PLAN` 的推进；
- **第二阶段（phase 2）**：目标状态属 `BUILD`、`VERIFY` 的推进，另含 `ocn task check`（仅限 `BUILD/VERIFY` 内）与里程碑回拨 `ocn rewind --to step_build_plan`（仅限自 `BUILD/VERIFY` 发起）；
- `PLAN`→`BUILD` 的跨阶段推进其目标状态属 `BUILD`，故归第二阶段：仅开启第一阶段时，该跨界推进仍应停下等待操作者；
- 目标状态属 `SHIP`、`REFLECT` 的推进永不委托。

被授权阶段内，AI 编码代理方可以 `OCN_ACTOR=ai_agent` 触发对应操作；未被授权的触发应被拒绝（退出码 4，`automation_not_enabled`）。

7.6.4 调用方身份

命令调用方身份由 `OCN_ACTOR` 环境变量解析（`--actor` 参数可显式覆盖），取值 `user` 或 `ai_agent`，写入审计记录。`ocn agent setup` 应将 `OCN_ACTOR=ai_agent` 注入 `.claude/settings.json`，使代理的全部命令自动携带该签名。该标记为治理签名而非安全边界：开关动作因开关本身拒绝 `ai_agent` 而始终可信。

7.6.5 决策痕迹

自动模式下 AI 编码代理的每次 `advance` 与 `task check` 应携带 `--rationale`（写明背景、依据与操作），缺失时应被拒绝（退出码 4，`automation_rationale_required`）；里程碑回拨复用其必填的 `--reason`。引擎应在审计中独立记录机器上下文（门禁结果、任务台账完成/未完成计数、冻结命令、执行耗时），不依赖代理自述。`ocn auto trace` 应自审计记录按时间线重放上述决策痕迹，供复盘。

7.6.6 熔断

AI 编码代理在同一步骤连续触发门禁失败达到阈值(automation.circuitBreaker.maxConsecutiveGateFailures 默认 5) 时, 引擎应自动暂停自动模式并拒绝该代理的后续受控操作 (退出码 4, automation_suspended), 同时写入 auto_mode_changed 暂停事件 (actor 为 system)。门禁通过、步骤变更或任一开关动作应清零失败计数。暂停期间操作者的操作不受影响。解除暂停应由操作者执行 ocn auto resume。熔断状态存于运行时文件 .ocoding/automation-runtime.json, 随 ocn cycle new 归档重置。

7.6.7 人工专属禁区

下列操作在任何模式下均不应委托给 AI 编码代理, 应拒绝 ai_agent 调用: ocn auto (开关本身)、ocn readiness waive、ocn cycle new、ocn rewind (除 7.6.3 所列里程碑回拨外)、ocn sop upgrade、advance 的门禁 override。MCP 接口不应因自动模式而新增任何工具——白名单维持既有 7 项工具不变。

7.6.8 触发前的独立评审

自动模式下无操作者逐步把关, 故 AI 编码代理在被授权阶段内行使任一受控触发 (ocn advance、ocn task check、里程碑回拨) 之前, 应先委派一个独立、全新上下文的评审子代理, 以资深领域专家视角, 依据本步骤要求、可追溯的验收准则、当次改动差异与本规范, 对实现给出 PASS/FAIL 结论与具体问题。该评审替代自动模式下被省略的人工评审, 其性质为尽职调查而非门禁: 门禁与冻结验收命令仍为唯一裁定, 评审结论既不放宽门禁, 亦不构成额外的硬阻断。

评审给出 FAIL 时, 应记录问题、在任务范围内修正、重新运行 ocn check 并复审; 修正次数上限为 3 次, 仍未通过时应将遗留问题写入触发命令的 --rationale (并经 ocn log 留痕) 后继续, 由门禁作最终裁定。评审结论仅覆盖当次改动; 改动未变更时可复用同一结论, 不应重复全量评审。

委派进程内评审子代理 (不经外部网络) 为本节要求的动作, 属治理性评审而非任务自身的外部服务调用。该要求仅在自动模式下适用; 手动模式由操作者亲自评审, 输出不含本项文案 (见 7.6.9)。

7.6.9 治理文案

ocn brief 与 ocn next-prompt 输出的治理段应随授权状态切换: 手动模式下与开启前逐字节一致; 某阶段被授权时, 应将“AI 不应推进状态”改述为该阶段的委托声明、并在 ocn next-prompt 的自动循环区块中前置 7.6.8 的独立评审步骤, 同时重申人工专属禁区; 熔断暂停时应标注并指向 ocn auto resume。

8 职责划分

8.1 AI 编码代理的允许行为

AI 编码代理可执行下列任务:

- 按结构模板生成文档初稿；
- 解析 git 与 GitHub 证据；
- 归纳当前状态；
- 识别缺失证据；
- 拆解任务；
- 生成下一轮任务提示词；
- 草拟验证汇总 (verification summary) 与判定草案 (verdict draft)；
- 解释状态；
- 在操作者按阶段开启自动模式后 (7.6)，于被授权阶段内以 OCN_ACTOR=ai_agent 自主触发状态推进 (ocn advance)、任务勾销 (ocn task check) 与里程碑回拨 (ocn rewind --to step_build_plan)；每次触发前应先完成触发前的独立评审 (7.6.8)，每次触发应携带决策理由 (--rationale)，并受门禁、熔断与人工专属禁区约束。

8.2 AI 编码代理的禁止行为

AI 编码代理不应执行下列操作：

- 自主发布 npm 包；
- 自主移动 latest 标签；
- 自主打 tag；
- 自主创建 release；
- 自主宣称 GA；
- 自主扩大修改范围；
- 在证据不足时给出确定性判定。

8.3 操作者保留职责

下列职责应由操作者保留，不应委托给 AI 编码代理：

- 范围裁定；
- 关键取舍；
- 风险接受；
- 最终验收裁定；
- 发布授权；
- merge 决策；
- 自动模式的开启、关闭与熔断解除 (ocn auto)，及由此对 AI 编码代理作出的阶段授权；
- 手动模式下的状态推进 (ocn advance) —— 开启自动模式即将被授权阶段内的推进触发委托给 AI (7.6)，但判定权不让渡；
- 重开循环与非里程碑回拨 (ocn cycle new / ocn rewind 除里程碑回拨外)；
- SOP 升级 (ocn sop upgrade)；

- 豁免授予 (ocn readiness waive)。

AI 编码代理为执行能力的放大器，不应作为责任主体；关键决策、验收裁定、风险承担、发布授权与产品取舍均应由操作者作出。

9 不符合项与纠正措施

实施本规范过程中检出下列不符合项时，应按对应纠正措施处置：

编号	不符合项描述	纠正措施
NC-01	定义未完成即进入实现：00 至 11 号文档未全部通过门禁而开始编码	返回第一阶段，按 5.1 补齐文档并通过门禁
NC-02	进入第二阶段后仍以文档推进为主要控制手段	转入 6.2 任务循环与 6.3 证据循环的交互模型
NC-03	手工重复记录 GitHub 已存在的证据	优先读取工程事实（6.1、6.4），停止重复转录
NC-04	以 AI 编码代理的输出作为最终裁定	AI 编码代理仅负责归纳与建议；裁定与授权由操作者作出（8.3）
NC-05	将 evidence-candidate 状态视同 evidence-found	维持保守判定；证据不足时不判定为完成（7.4）
NC-06	verify status 为 partial 时执行 merge	先补证据、再补验证、确认 ready 后方可进入 review 与 merge（7.4）
NC-07	文档结构齐全但逻辑未接线：模块、指标、公式齐备，而运行时计算/决策语义未显式定义	在设计层产出逻辑主干（5.2），将执行顺序、公式归属、分数分层、信号作用级别建为可机器校验的图；六类硬缺陷未清零不应进入 BUILD
NC-08	角色盲区：各门禁通过，但缺少版本控制/CI、可复跑测试命令、使用者或可运维性责任人	执行 ocn readiness list 检视当前 tier 下 FAIL 与 UNKNOWN 项，按 fix_hint 补证据；确实不适用的检查按 5.4.5 显式豁免并留痕。无证据不视为通过
NC-09	实现缺位与全链空转：BUILD 阶段过程文档如实记录“无代码变更”而门禁全部通过，状态推进持续进行	升级至 SOP 0.5.0，启用任务主干（5.3）：build plan 拆分任务规格，完成判定仅认冻结的验收命令，台账未清不应离开 state_build。已空转至 VERIFY 中段时：以 ocn rewind --to step_build_plan --reason <text> 将游标受控回拨至 build plan，补齐任务规格并重新通过门禁以生成任务台账，再按任务循环逐项完成。状态推进不构成义务（7.3）
NC-10	直接编辑 .ocoding/state.json 移动状态游标	改用受控命令：轮内回退使用 ocn rewind，跨轮重开使用 ocn cycle new（7.3）；两者均持锁写入并产生审计记录，使游标移动可解释、可追溯

附录 A（规范性） 操作规程

A.1 安装、升级与卸载

本规范由 OCN 落地。开工前应安装 ocn 与 ocn-mcp 两个命令。前置依赖：Node.js ≥ 20 。

安装（npm 全局）：

```
npm install -g o-coding-navigation    # latest 通道，当前对应 SOP 0.5.0
ocn --version                        # 验证安装；预期输出 0.6.0-beta.0
ocn-mcp                             # 验证 MCP 服务器可启动；Ctrl+C 退出
```

如需固定 beta 预发布通道：npm install -g o-coding-navigation@beta。

贡献者可采用源码路径：git clone https://github.com/UncleTIM-GZ/O-CodingNavigation.git
&& cd O-CodingNavigation && npm install && npm run build && npm link。

升级分两层，应全部执行：

```
# 第一层：升级工具本体（npm 包）
npm install -g o-coding-navigation@latest

# 第二层：升级既有项目锁定的 SOP 版本（在项目目录内执行）
ocn sop upgrade --plan    # 干跑：列出将要改动的快照文件
ocn sop upgrade           # 执行：迁移到当前内置默认版本（0.5.0）
```

仅执行第一层而不执行第二层时，既有项目仍按初始化时锁定的旧 SOP 运行；该行为属于设计行为。第二层升级中，进度游标、已写文档与 config.yaml 自定义命令应全部保留。仅支持向前升级。

卸载：

```
npm uninstall -g o-coding-navigation    # 卸载工具
rm -rf .ocoding                        # 可选：删除某项目的 OCN 状态；docs/ 下文档不受影响
```

完整安装步骤、ocn init 后的文件树与常见排障见 docs/quickstart.md。

A.2 项目初始化（一次性）

```
cd <项目目录>
ocn init --tier minimal    # 创建 .ocoding/ 与 docs/，锁定 SOP 0.5.0
ocn status                 # 确认当前位置：state_discovery / step_project_brief
```

初始化后应立即在 .ocoding/config.yaml 的 commands：段登记本项目的构建与测试命令。就绪检查依赖该登记取证；未登记时相关检查恒为 UNKNOWN，而 UNKNOWN 阻断（5.4.4）：

```
commands:
  build: npm run build
  test: npm run test
  test_list: npx vitest list
```

随后应执行一次就绪基线评估，确认距离开工条件尚缺哪些角色的证据：

```
ocn readiness list
```

最后应接线 Claude Code，生成全部集成文件并提交入库（行为定义见 7.5）：

```
ocn agent setup # 幂等；--force 仅用于修复损坏的 settings.json
git add .claude CLAUDE.md && git commit # 入库后，克隆仓库的成员无须单独配置
```

A.3 第一阶段规程（00 至 11）

每份文档应执行同一个四步循环。已接线 Claude Code 时：输入 /ocn-next 自动完成第 1、2 步并开始执行，第 4 步由 Stop 钩子在代理结束回合时强制运行。

```
ocn brief # 第 1 步：查看当前步骤的产物要求与必需章节
ocn doc create <type> # 第 2 步：从模板创建文档
#（操作者与 AI 编码代理填写内容） # 第 3 步：填实内容
ocn check # 第 4 步：三道门禁——章节、逻辑主干、就绪
ocn advance # 通过后推进到下一步（advance 自动先行运行门禁）
```

<type> 取值按顺序如下：

序号	文档	ocn doc create <type>
00	项目简报	project-brief
01	范围	scope
02	PRD	prd
03	验收标准	acceptance-criteria
04	技术架构	technical-architecture
05	信息架构	information-architecture
06	数据模型	data-model
07	逻辑主干	logic-backbone
08	接口契约	api-contract
09	测试策略	test-strategy
10	MVP 计划	mvp-plan
11	Build Plan	build-plan

被 ocn check 阻断时应按退出码处置（语义见 7.2）：

- **退出码 2**（产物问题）：缺必需章节，07 号逻辑主干命中六类硬缺陷（缺角色、重复 id、悬空引用、依赖环、孤儿节点、未绑定触发），或 11 号 build plan 的任务规格命中六类硬缺陷（重复或非法 id、缺字段、traces 悬空、touches 悬空、depends 悬空或成环、零任务）——应修订文档本身；
- **退出码 1**（门禁问题）：就绪检查未通过——应执行 `ocn readiness list` 查看 FAIL 与 UNKNOWN 的角色检查，按 `fix_hint` 补充证据，或在确实不适用时显式豁免（A.4）。

11 号 build plan 门禁通过时，任务台账 `.ocoding/task-ledger.json` 生成、验收命令冻结——第一阶段结束，进入第二阶段的任务循环。

A.4 就绪检查时机

就绪检查已内嵌于每次 `check`、`gate`、`advance`，无须另行安排。需要主动执行的时机仅有下列四个：

时机	命令
初始化之后，建立基线	<code>ocn readiness list</code>
任何 <code>check</code> 或 <code>advance</code> 以退出码 1 被阻断时	<code>ocn readiness list</code> （查看阻断项与 <code>fix_hint</code> ）
某条检查确实不适用于本项目时	<code>ocn readiness waive <checkId> --reason "..."</code> <code>--probe "..."</code>
进入 BUILD（11 号之后）前的最终核查	<code>ocn readiness list</code>

豁免示例：

```
ocn readiness waive rdy_network_engineer \  
  --reason " 纯本地 CLI 工具，无网络架构" \  
  --probe "test ! -d infra"
```

豁免语义（开放世界判定、授予即验证并持续复验、状态切换即过期、操作者专属且写入审计）以 5.4.5 为准。既有项目迁移见 A.1。

A.5 第二阶段规程（11 号之后）

第二阶段不以 `advance` 为主要交互方式，改为任务循环与证据循环。自 SOP 0.5.0 起，BUILD 状态的核心节奏为逐个完成任务台账中的任务：

```
ocn task list                # 查看台账：各任务状态与下一任务  
/ocn-next                   # 派发第一个未完成任务（目标为任务规格原文）  
#（AI 编码代理按测试驱动方式实现，范围限于该任务的 touches）  
ocn task check              # 执行该任务冻结的验收命令；退出码 0 方确立完成  
#（重复以上循环直至台账全清；台账未清时 advance 不放行）
```

证据循环的一轮典型迭代如下（各命令功能定义与使用时机见 6.4）。已接线 Claude Code 时：
/ocn-next 一步替代第 1 至第 4 步；第 5 步期间的编辑反馈与回合结束门禁由钩子自动执行；
第 6、7 步为操作者收口工具，任何模式下均不被替代：

```
ocn exec status          # 第 1 步：读取本地 git 现场（分支、脏/净、改动文件）
ocn github analyze-pr <n> # 第 2 步：存在 PR 时读取 PR 元数据、checks、reviews
ocn evidence map [--pr <n>] # 第 3 步：对照 03 验收标准映射证据
ocn next-prompt          # 第 4 步：生成下一轮 AI 编码代理任务简报
# (AI 编码代理执行一轮实现) # 第 5 步
ocn verify status --mode combined --pr <n> # 第 6 步：汇总验证就绪度 ready/partial/blocked/
↔ pending
ocn verdict draft        # 第 7 步：草拟阶段判定，交由操作者裁定
```

A.6 命令速查

```
# 安装
npm install -g o-coding-navigation && ocn --version

# 初始化
ocn init --tier minimal
# (编辑 .ocoding/config.yaml, 登记 commands.build/test/lint/typecheck)
ocn readiness list          # 就绪基线
ocn agent setup            # 接线 Claude Code (钩子、治理契约、/ocn-next)
git add .claude CLAUDE.md && git commit # 入库共享

# 第一阶段 (对 00 至 11 每份文档重复四步循环)
ocn brief
ocn doc create <type>
# (填写内容)
ocn check
ocn advance
# 已接线时: Claude Code 中 /ocn-next 自动执行前两步, Stop 钩子强制运行 check
# 退出码 2: 修订章节、逻辑主干或任务规格; 退出码 1: ocn readiness list 后补证据或豁免
# 11 号 build plan 应含 Task Specs 拆分; 门禁通过即冻结任务台账

# 第二阶段 (每轮迭代重复)
# 读现场与派发, 二者选一:
# 手动: ocn exec status; ocn evidence map; ocn next-prompt; 将简报交给代理
# 已接线: /ocn-next 一步替代
# BUILD 状态任务循环 (SOP 0.5.0 核心节奏):
ocn task list
# (/ocn-next 派发任务; AI 编码代理实现)
```

```

ocn task check
# 台账未清时 advance 不允许离开 state_build
# 收口（操作者裁定，不被 /ocn-next 替代）：
ocn verify status
ocn verdict draft
# （操作者 review 后执行）
ocn advance

# 回拨与重开（操作者专属，不暴露给 AI 编码代理；用于里程碑衔接与返工两类场景）
ocn rewind --to <stepId> --reason "<text>"
# 轮内回拨；零豁免，回拨后 advance 重新运行全部门禁
ocn cycle new --yes
# 终点步骤后开启新一轮；文档保留，审计记录跨轮连续

# 自动模式（可选，默认关闭；开关为操作者专属，见 7.6）
ocn auto on --phase 1 # 委托第一阶段（DISCOVERY→PLAN）
ocn auto on --phase all # 全自动：规划 + BUILD/VERIFY 循环 + 里程碑回拨
ocn auto status # 查看当前授权与熔断状态
ocn auto trace # 复盘每条 AI 决策（理由 + 机器上下文）
ocn auto resume # 熔断后解除暂停
ocn auto off # 回到全手动
# 开启后 AI 以 OCN_ACTOR=ai_agent + --rationale 触发 advance / task check / 里程碑 rewind
# 门禁与冻结验收命令照常判定；熔断与人工专属禁区始终生效

# 升级
npm install -g o-coding-navigation@latest && ocn sop upgrade

# 卸载
npm uninstall -g o-coding-navigation

```

A.7 自动模式规程（可选）

自动模式默认关闭；下列规程仅在操作者决定按阶段委托推进触发时适用。开关为操作者专属（拒绝 ai_agent），判定权不随开启而让渡（语义见 7.6）。

开启与查看（操作者执行）：

```

ocn auto on --phase 1 # 仅委托第一阶段；--phase 取 1 | 2 | all，必填
ocn auto status # 确认当前授权与熔断状态

```

被授权阶段内，AI 编码代理在每次触发前应先委派独立评审子代理校验实现（7.6.8），并使触发携带身份签名与决策理由（已接线 Claude Code 时，ocn agent setup 已将 OCN_ACTOR=ai_agent 注入 settings，无须逐条添加）：

```
OCN_ACTOR=ai_agent ocn advance --rationale " 背景:…; 依据: 门禁全绿; 操作:advance"
OCN_ACTOR=ai_agent ocn task check --rationale " 背景: 任务 X; 依据: 冻结命令 exit 0; 操
↪ 作:check"
# 多里程碑项目: 完成一个里程碑后由 AI 自驱回拨续接 (仅此一种回拨可委托)
OCN_ACTOR=ai_agent ocn rewind --to step_build_plan --reason "P0 完成, 追加 P1 任务"
```

复盘、熔断解除与关闭（操作者执行）：

```
ocn auto trace                # 按时间线重放每条 AI 决策（理由 + 机器上下文）
ocn auto resume               # 连续门禁失败触发熔断后，解除暂停
ocn auto off                  # 撤销委托，回到全手动
```

自动循环的机器停机条件：触发被拒（reason 以 automation_ 起始）、熔断暂停、推进目标越出被授权阶段、到达终点步骤且无剩余里程碑——遇任一条件，AI 应停止并交还操作者。

附录 B（资料性）修订历史

版本	日期	变更摘要
v1	2026-05-05	初版：在原线性文档 SOP 基础上确立两阶段模型（Planning Gate / Execution Navigator），将开发前的不确定性问题转换为结构化定义问题，并以工程证据链作为第二阶段事实来源
v2	2026-05-05	两阶段模型同日修订与首个排版发布版
v3	2026-06-04	新增逻辑主干（Logic Backbone）：设计层强制产物，将计算/决策语义建为可机器校验的有向图，判定第二类虚假完成；对应 OCN 0.3.0 artifact_logic_backbone
v4	2026-06-10	可读性整理：重复表述归并至唯一规范条目；新增安装指南章。方法论不变
v5	2026-06-11	新增就绪主干（Readiness Backbone）：基于 54 个 IT 角色编目的 55 条可证伪就绪检查，作为横切门禁贯穿两阶段，判定第三类虚假完成；对应 OCN 0.4.0 readiness 门禁，配套 ocn sop upgrade 迁移。新增完整操作指引章，安装指南并入其中
v6	2026-06-12	Claude Code 集成产品化：ocn agent setup 一次生成钩子、治理契约与 /ocn-next 斜杠命令，回合结束门禁与编辑即时反馈由钩子机械执行；对应 OCN 0.4.0-beta.2 的 ocn agent setup 与 ocn hook 命令
v7	2026-06-12	新增任务主干（Task Backbone）：build plan 内嵌机器可解析的任务规格块，门禁校验六类硬缺陷并将验收命令哈希冻结进任务台账，完成判定仅认验收命令、台账未清不离开 BUILD，判定第四类虚假完成（含全链空转）；对应 OCN 0.5.0 task 门禁与 ocn task 命令。至此四类虚假完成均有机械判定机制
1.0	2026-06-12	由 v7 转换为工程过程规范文体（OCN-SPEC-001）：第三人称声明式行文，统一规范用语（应/不应/宜/可），术语先行定义，常见错误改写为不符合项与纠正措施。技术内容与 v7 等同，未新增或删除功能要求

版本	日期	变更摘要
1.1	2026-06-12	新增状态回拨（ocn rewind）与重开循环（ocn cycle new）要求（对应 OCN 0.5.0-beta.1 / DEC-033）：术语 3.22、3.23；推进规则补充回退与重开条款（7.3）；操作者保留职责增列（8.3）；不符合项新增 NC-10（直接编辑 state.json），NC-09 纠正措施更新为受控回拨路径；附录 A.6 命令速查同步
1.2	2026-06-12	新增里程碑循环条款（7.3）：多里程碑项目宜以回拨循环衔接各里程碑——回拨至 build plan、保留既有任务规格原文并追加下一里程碑任务、重过门禁；任务台账按哈希对账保留已完成状态，构成项目级累积台账；重开循环限定于完成边界达成后使用。经 Lattice dogfood 实战验证（19 done + 7 pending 累积台账）；引擎行为审计确认无缺失
1.3	2026-06-12	术语校正：附录 A.6 的“纠错”标签更正为“回拨与重开”——回拨的用途包括里程碑衔接（常规节奏，见 7.3 里程碑循环条款）与返工（异常恢复）两类，“纠错”仅覆盖后者。无功能性变更
1.4	2026-06-13	新增可选自动模式（auto mode，对应 OCN 0.6.0-beta.0 / AM-009 / DEC-034）：操作者可按阶段将状态推进与任务勾销的触发委托给 AI 编码代理，委托触发权而非裁定权——门禁栈与冻结验收命令照常判定。术语 3.24–3.27（自动模式、调用方身份、熔断、决策痕迹）；过程模型补充阶段授权说明（4.3）；新增第 7.6 节（授权开关、阶段委托范围、调用方身份、决策痕迹、熔断、人工专属禁区、治理文案）；职责划分更新（8.1 增列受权触发行为，8.3 将推进改述为“手动专属、开启后委托但判定权不让渡”，新增 ocn auto 与 ocn sop upgrade 为操作者保留）；附录新增 A.7 自动模式规程并同步 A.6 命令速查。引擎/CLI 特性，不 bump SOP 版本；MCP 白名单 7 工具不变
1.5（本规范）	2026-06-15	新增自动模式触发前的独立评审要求（对应 AM-011 / DEC-036）：自动模式下无人工逐步把关，AI 编码代理在被授权阶段内行使任一受控触发（advance / task check / 里程碑回拨）前，应先委派独立、全新上下文的评审子代理以资深专家视角校验实现，给出 PASS/FAIL；该评审为替代被省略人工评审的尽职调查，裁定权仍属门禁与冻结验收命令。FAIL 时记录—修正—复审，上限 3 次，仍不过则写入 --rationale 后继续；结论仅覆盖当次改动。新增术语 3.28（独立评审）；新增 7.6.8（触发前的独立评审），原 7.6.8 治理文案顺延为 7.6.9 并补充自动循环区块前置评审步骤；职责划分 8.1 增列触发前评审；附录 A.7 同步。引擎/CLI 文本层特性，不 bump SOP 版本；MCP 白名单 7 工具不变